



Technologies: Python 3.11 | NetworkX | Streamlit | Pyvis | pytest
Algorithms: A* Search | Bayesian Inference | Minimax/Alpha-Beta | CSP

Repository: github.com/surya-8143/FOAI

. Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. PEAS Formulation
4. Environment Analysis
5. Network Topology & Graph Structure
6. Red Agent - A* Search Algorithm
7. Blue Agent 1 - Monitor (Bayesian Belief Tracking)
8. Blue Agent 2 - Response (Minimax + Alpha-Beta + Expectimax)
9. Blue Agent 3 - Patch Scheduler (CSP)
10. Multi-Agent Coordination (Message Bus)
11. Simulation Engine & Tick Loop
12. Live Dashboard (Streamlit + Pyvis)
13. Demo Scenarios & Test Suite
14. Metrics & Performance Analysis
15. Code Structure & Scalability
16. Technology Stack
17. How to Run
18. Conclusion

1. Executive Summary

SentinelNet is a multi-agent adversarial network defense simulator designed as a case study for the Foundations of Artificial Intelligence course. The system models a simulated enterprise network as a directed graph of servers, databases, and workstations under attack by a Red agent (attacker) whose goal is to reach a high-value "Crown Jewel" node. A team of three Blue defender agents collaborates to detect, isolate, patch, and recover network assets.

The project demonstrates four distinct AI algorithms - A* Search for attack path planning, Bayesian Belief Update with Spatial Diffusion for threat monitoring under uncertainty, Minimax with Alpha-Beta Pruning and Expectimax for adversarial decision-making, and Constraint Satisfaction Problem (CSP) solving with Backtracking, MRV heuristic, and Forward Checking for patch scheduling. All simulation is abstract - no real exploits or network scanning are performed.

The system features a live Streamlit dashboard with real-time network graph visualization (Pyvis), belief probability charts, action logs, and performance metrics. Five distinct demo scenarios cover baseline comparison, stealthy attackers, multi-entry attacks, resource-starved defense, and scalability testing (20 to 500 nodes). The full test suite of 57 tests passes.

2. Problem Statement & Motivation

2.1 The Challenge

Enterprise networks face persistent, evolving cyber threats. Attackers use sophisticated pathfinding strategies to navigate through network layers and reach critical assets. Defenders must detect attacker presence through noisy, imperfect sensors and respond with limited resources under uncertainty.

2.2 Our Approach

SentinelNet abstracts this problem into a tick-based multi-agent simulation where:

- A Red agent (attacker) navigates the network graph using A* search
- Three Blue agents (defenders) cooperate to protect the Crown Jewel
- Communication happens through a shared pub/sub Message Bus (blackboard architecture)
- Each agent uses a different AI algorithm justified by the specific problem feature it addresses

2.3 Key Research Question

Can coordinated AI defender agents protect a critical network asset against an intelligent attacker operating under conditions of partial observability, stochastic exploit outcomes, and dynamic network state changes?

3. PEAS Formulation

The PEAS framework defines the agent's Performance measure, Environment properties, Actuators (actions), and Sensors (inputs). We formulate PEAS for the Blue team agents:

3.1 Performance Measure

The Blue team is evaluated using a composite utility function:

$$\text{Utility} = 10 \times \text{uptime_fraction} - 5 \times \text{compromised_value} - 0.5 \times \text{response_cost}$$

- uptime_fraction: Percentage of nodes that are operational (not isolated or compromised)
- compromised_value: Weighted sum of asset values of compromised nodes
- response_cost: Number of defensive actions taken (isolate=1, patch=2, restore=3)
- Time-to-Detection (TTD): Ticks until Monitor correctly identifies Red's location

3.2 Environment

Type	Description
Observability	Partially Observable - Blue sees noisy IDS alerts, not Red's true location
Determinism	Stochastic - Exploit success = vuln_score probability (0.0 to 1.0)
Dynamism	Dynamic - Red moves independently each tick, network state changes
Agents	Multi-Agent - Competitive (Red vs Blue) + Cooperative (Blue team)
State Space	Discrete - Finite nodes, edges, states, and action sets
Episodic/Sequential	Sequential - Tick-by-tick decisions affect future state

3.3 Actuators (Blue Team Actions)

Action	Effect	Cost
ISOLATE	Disconnect node from graph (blocks Red traversal)	1
PATCH	Reduce vuln_score to near zero (takes 1 tick)	2
RESTORE	Bring isolated/compromised node back online	3
DEPLOY HONEYPOT	Set trap node to capture Red agent	1

3.4 Sensors

- IDS Alerts: Probabilistic intrusion detection events
 - True positive: $P(\text{alert} \mid \text{Red at node}) = \text{detection_rate}$ (default 0.7)
 - False positive: $P(\text{alert} \mid \text{Red NOT at node}) = \text{false_alarm_rate}$ (default 0.05)
- Node State Queries: Direct query of node state after actions
- Belief Distribution: Posterior probability map from Monitor agent

4. Environment Analysis

The simulation environment exhibits rich properties that justify the choice of algorithms:

4.1 Partial Observability

Blue agents cannot directly observe Red's location. The IDS sensor provides noisy alerts with both false positives ($\text{false_alarm_rate} = 0.05$) and missed detections ($1 - \text{detection_rate} = 0.3$). This uncertainty necessitates Bayesian belief tracking to maintain a probability distribution over possible attacker locations.

4.2 Stochastic Transitions

When Red attempts to exploit a node, success is probabilistic: $P(\text{success}) = \text{vuln_score}$. A node with $\text{vuln_score} = 0.3$ has only a 30% chance of being compromised per attempt. This stochasticity motivates the use of Expectimax over pure Minimax, as Red does not have deterministic control over exploit outcomes.

4.3 Adversarial + Cooperative Dynamics

The system is both competitive (Red vs Blue) and cooperative (Blue agents share information via the Message Bus). The competitive aspect justifies Minimax game-tree search, while the cooperative aspect requires a coordination mechanism (blackboard pattern).

4.4 Dynamic State Changes

The network state changes every tick: Red may compromise new nodes, Blue may isolate or patch nodes, and the patch scheduler modifies vulnerability scores. This dynamic nature requires Red's A* planner to support dynamic replanning when paths are blocked.

5. Network Topology & Graph Structure

5.1 Graph Representation

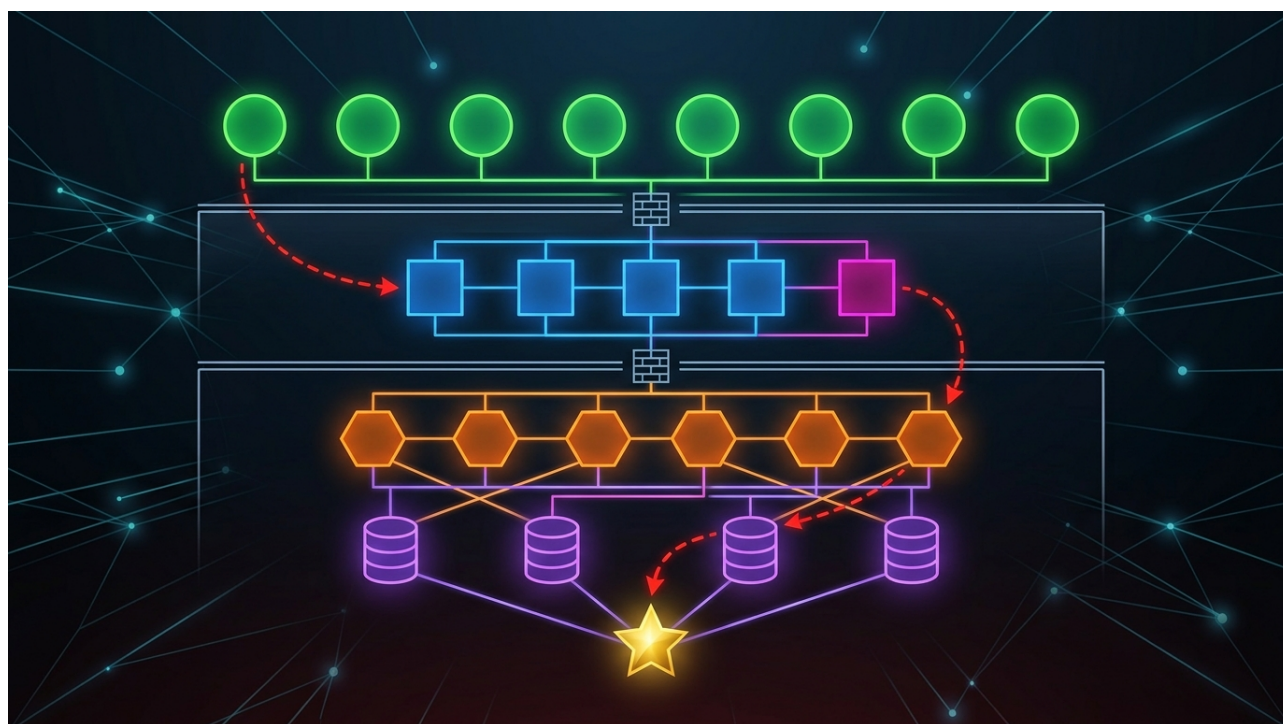
The enterprise network is modeled as a NetworkX DiGraph (directed graph). Each node represents a network asset and carries a NodeData dataclass with attributes:

Attribute	Description
name	Unique node identifier (e.g., ws0, srv_dmz1, crown_jewel)
node_type	One of: workstation, server, database, crown_jewel
value	Asset value (workstation=1-2, server=4-8, database=8-10, crown_jewel=20)
vuln_score	Exploit probability [0.0, 1.0] (perimeter=0.5-0.75, internal=0.2-0.35)
state	SAFE SUSPECTED COMPROMISED ISOLATED PATCHING HONEYPOT
patched	Boolean; if True, effective_vuln = vuln_score * 0.1
services	List of running services (e.g., ssh, http, mysql)

5.2 20-Node Demo Topology (network_small.yaml)

The hand-crafted 20-node topology models a realistic enterprise network with 5 layers:

Layer 1: Perimeter Workstations	ws0-ws5 (6 nodes, vuln 0.50-0.75)
Layer 2: DMZ Servers	srv_dmz0-1 (2 nodes, vuln 0.40-0.45)
Layer 3: Internal Servers	srv_int0-2, srv_app0-1 (5 nodes, vuln 0.20-0.35)
Layer 4: Databases	db0-db2 (3 nodes, vuln 0.12-0.18)
Layer 5: Crown Jewel	crown_jewel (1 node, vuln 0.05)
Special: Honeypot	honeypot0 (1 node, vuln 0.90 - trap!)
Management	mgmt0-1 (2 nodes, vuln 0.28-0.32)



5.3 Edge Costs

Edges carry a cost attribute (1-5) representing traversal difficulty. Red's A* uses these costs for pathfinding. Blue isolation removes traversable edges. The honeypot has cost=1 edges to attract Red into the trap.

5.4 Scalable Topologies

Medium (100 nodes) and large (500 nodes) topologies are generated procedurally using `NetworkGraph.generate_random()`. The generator creates a Barabasi-Albert-like connected graph with 60% workstations, 25% servers, 14% databases, and 1 crown jewel. Extra shortcut edges are added ($n/2$) for realism.

6. Red Agent - A* Search Algorithm

6.1 Algorithm Overview

The Red agent uses A* search (Hart, Nilsson & Raphael, 1968) to find the optimal least-cost path from its entry point to the Crown Jewel node. A* is chosen because it is optimal and complete when using an admissible heuristic, and it efficiently guides the search toward the goal using the evaluation function $f(n) = g(n) + h(n)$.

6.2 Evaluation Function

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual traversal cost from entry point to node n
(sum of edge costs along the path so far)

$h(n)$ = admissible heuristic estimate of remaining cost to crown_jewel
(BFS hop-count on the unweighted reverse graph)

Properties:

- Admissible: $h(n) \leq \text{true cost}$ (each edge costs ≥ 1)
- Consistent: $h(n) \leq \text{cost}(n, n') + h(n')$ for all edges

6.3 BFS Heuristic Factory

The heuristic is precomputed using a backward BFS from the Crown Jewel. For each node, it returns the unweighted hop count, which is always $\leq$ the true weighted cost (since all edge costs ≥ 1). This makes the heuristic both admissible and consistent.

```
def bfs_heuristic_factory(graph, goal):
    dist = {} # node -> hop count
    queue = deque([(goal, 0)])
    while queue:
        n, d = queue.popleft()
        if n in dist: continue
        dist[n] = d
        for pred in graph.g.predecessors(n):
            if pred not in dist:
                queue.append((pred, d + 1))
    return lambda node, _: float(dist.get(node, 0))
```

6.4 A* Implementation

The core A* function uses a min-heap priority queue:

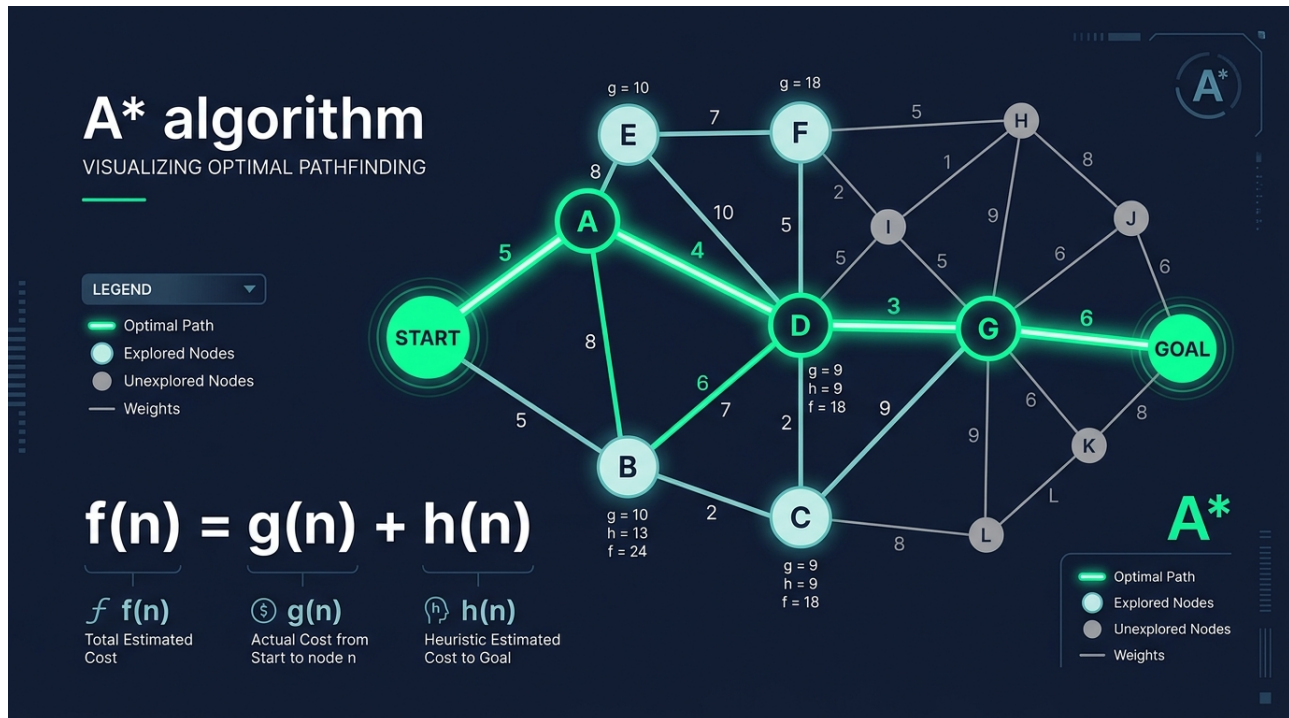
```
def a_star(graph, start, goal, heuristic):
    frontier = [(h(start), 0.0, start, [start])] # (f, g, node, path)
    best_g = {}
    nodes_expanded = 0

    while frontier:
        f, g, node, path = heappop(frontier)
        nodes_expanded += 1
        if node == goal:
```

```

    return path, g, nodes_expanded
    if node in best_g and best_g[node] <= g:
        continue
    best_g[node] = g
    for neighbor, edge_cost in graph.neighbors_with_cost(node):
        new_g = g + edge_cost
        heappush(frontier, (new_g + h(neighbor), new_g, neighbor, path+[neighbor]))
    return None, inf, nodes_expanded

```



6.5 Dynamic Replanning

If Blue isolates a node on Red's planned path, Red detects this when attempting to move (the node state is ISOLATED). Red then re-invokes A* from its current position to find an alternative route. This replanning is critical for the adversarial dynamic.

6.6 IDS Alert Generation

Red generates noisy alerts for the Blue team's Monitor agent:

- True positive: With $P = \text{detection_rate}$ (0.7 default), alert fires on the actual target node
- False positive: With $P = \text{false_alarm_rate}$ (0.05 default), alert fires on a random other node
- In stealth mode (Scenario 2): detection_rate is halved to 0.35
- Honeypot detection: If Red enters a honeypot node, a high-confidence (0.99) alert fires and Red is captured

7. Blue Agent 1 - Monitor (Bayesian Belief Tracking)

7.1 Problem

The Monitor agent never directly observes Red's location. It receives noisy IDS alerts and must maintain a probability distribution over all network nodes estimating where Red most likely is. This is a classic partially observable inference problem.

7.2 Bayesian Update Step

When an alert fires at node X, Bayes' theorem is applied to every node:

```
P(Red at X | alert at X) = detection_rate x prior(X)    [likelihood for alert node]
P(Red at Y | alert at X) = false_alarm_rate x prior(Y)  [likelihood for other nodes]
```

Then normalise: $P(n) = P(n) / \text{sum}(P(\text{all nodes}))$
so all probabilities sum to 1.0

If no alert fires: belief unchanged (attacker may be hiding)

```
def bayesian_update(belief, alert_node, detection_rate, false_alarm_rate, graph):
    if alert_node is None:
        return dict(belief)
    new_belief = {}
    for node, prior in belief.items():
        likelihood = detection_rate if node == alert_node else false_alarm_rate
        new_belief[node] = prior * likelihood
    total = sum(new_belief.values())
    if total < 1e-12: # Underflow guard
        return {node: 1.0/len(new_belief) for node in new_belief}
    return {node: p/total for node, p in new_belief.items()}
```

7.3 Spatial Diffusion Step

After the Bayesian update, a diffusion step models Red's possible movement between ticks. This prevents the belief from becoming overly concentrated on a single node:

```
P(Red stays at current node)      = stay_prob (0.6)
P(Red moves to each neighbor)     = (1 - stay_prob) / N_neighbors
```

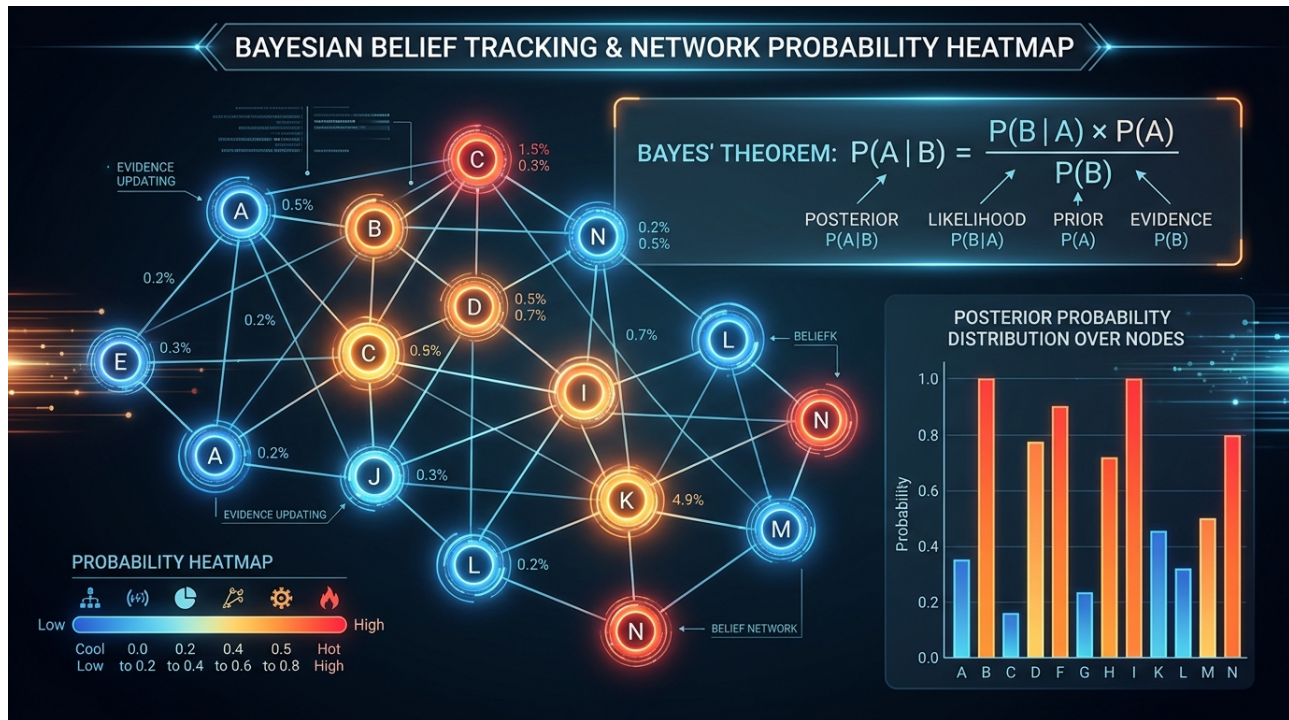
```
For each node n with belief P(n):
    new_belief[n] += P(n) * 0.6
    for each neighbor nb:
        new_belief[nb] += P(n) * 0.4 / |neighbors(n)|
Renormalise to sum to 1.0
```

7.4 Particle Filter (Large Graphs)

For graphs with >200 nodes, a particle filter variant is available. Instead of maintaining a dense probability dict, it tracks N=500 weighted particles (node positions). Each tick:

- Transition: Each particle stays (P=0.6) or moves to a random neighbor (P=0.4)
- Observation: Weights updated by likelihood given alert

- Resample: Low-variance resampling to prevent particle degeneracy



7.5 Output

The Monitor publishes the updated belief dictionary to the 'belief' topic on the Message Bus. The Response agent reads this to focus its game-tree search on the top-k most suspected nodes. The dashboard displays the top-5 beliefs as a bar chart.

8. Blue Agent 2 - Response (Minimax + Alpha-Beta + Expectimax)

8.1 Game-Tree Decision Making

The Response agent models the Red-vs-Blue interaction as a two-player zero-sum game. It uses depth-limited game tree search to evaluate defensive actions and select the one that maximizes Blue's utility under worst-case (Minimax) or expected-case (Expectimax) Red behavior.

8.2 GameState Snapshot

To avoid deep-copying the full NetworkX graph at every game tree node, a lightweight GameState dataclass captures only the minimum information needed for the utility function:

```
@dataclass
class GameState:
    node_states: Dict[str, str]      # {node_id: state_value}
    node_values: Dict[str, int]      # {node_id: asset_value}
    node_patched: Dict[str, bool]    # {node_id: patched}
    node_vulns: Dict[str, float]     # {node_id: effective_vuln}
    crown_jewel: str
    total_action_cost: int = 0
```

8.3 Utility Function

Blue's utility is a weighted combination of three objectives:

```
utility = w_uptime * uptime_fraction
        - w_compromise * compromised_value
        - w_cost * total_action_cost

Default weights: w_uptime=10, w_compromise=5, w_cost=0.5

uptime_fraction = (total_nodes - isolated - compromised) / total_nodes
compromised_value = sum(value[n] for n if state[n] == COMPROMISED)
total_action_cost = cumulative actions taken
```

8.4 Minimax with Alpha-Beta Pruning

The Minimax algorithm models a worst-case adversary. Blue (MAX) chooses the action that maximizes utility; Red (MIN) is assumed to always succeed and choose the worst outcome for Blue. Alpha-beta pruning eliminates branches that cannot affect the final decision:

```
def minimax(state, depth, alpha, beta, maximizing, top_suspects):
    if depth == 0 or state.is_terminal():
        return utility(state), None

    if maximizing: # Blue's turn
        value = -inf
        for action in blue_actions(state, top_suspects):
            child = apply_blue_action(state, action)
            score, _ = minimax(child, depth-1, alpha, beta, False, top_suspects)
            if score > value: value, best = score, action
            alpha = max(alpha, value)
```

```

    if alpha >= beta: break # Beta-cutoff (prune)
    return value, best

else: # Red's turn (worst case)
    value = +inf
    for node, _ in red_actions(state, top_suspects):
        child = apply_red_action(state, node, success=True)
        score, _ = minimax(child, depth-1, alpha, beta, True, top_suspects)
        if score < value: value = score
        beta = min(beta, value)
        if alpha >= beta: break # Alpha-cutoff (prune)
    return value, None

```

Complexity: Without pruning $O(b^d)$; with alpha-beta $O(b^{(d/2)})$

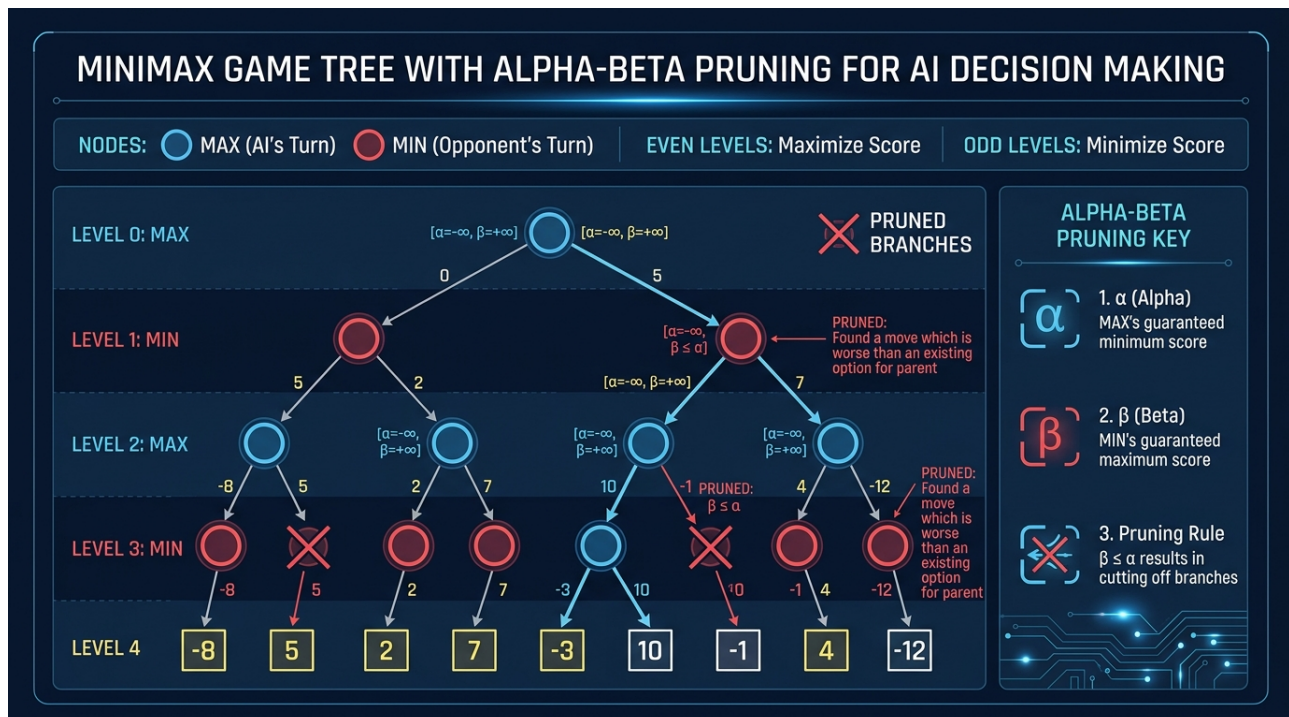
8.5 Expectimax Enhancement

Expectimax replaces the deterministic MIN node with a CHANCE node. Instead of assuming Red always succeeds, it averages over the exploit probability:

$$E[\text{value}] = \text{vuln_score} \times V(\text{success}) + (1 - \text{vuln_score}) \times V(\text{failure})$$

Red's turn becomes a weighted average over all possible target nodes, each with success/failure branches weighted by exploit probability.

This is more realistic because Red cannot control whether an exploit works. The Response agent defaults to Expectimax (use_expectimax=True).



8.6 Integration with Monitor

The key integration point: Response reads the Monitor's belief and extracts the top-k (default k=5) most-suspected nodes. Only these nodes are considered as possible Red locations in the game tree, keeping the branching factor tractable.

9. Blue Agent 3 - Patch Scheduler (CSP)

9.1 Problem Formulation

While the Response agent handles immediate threats, the Patch Scheduler manages long-term vulnerability reduction. It frames the scheduling of patch jobs as a Constraint Satisfaction Problem (CSP).

9.2 CSP Components

Component	Description	Example
Variables	One per pending patch job (vulnerable node)	ws0, srv_int1, db2
Domains	Available maintenance time slots	{0,1,2,...,9}
C1: Window	Job must run within its maintenance window	slot(ws0) in {2,3,4,5}
C2: Ordering	Dependencies: slot(A) < slot(B)	slot(srv_int0) < slot(db0)
C3: AllDiff	Same technician, different slots	slot(ws0) != slot(ws1) if same tech

9.3 Hand-Rolled CSP Solver

The solver implements three key techniques from scratch (no library):

9.3.1 Backtracking Search

Systematic depth-first search that assigns values to variables one at a time. If a constraint is violated, it backtracks to the previous variable and tries the next value in its domain.

9.3.2 MRV (Minimum Remaining Values) Heuristic

At each step, select the unassigned variable with the fewest remaining valid values in its domain. This finds conflicts early and dramatically reduces the search space.

```
def _select_unassigned(self, assignment):
    unassigned = [v for v in self.variables if v not in assignment]
    return min(unassigned, key=lambda v: len(self.domains[v]))
```

9.3.3 Forward Checking

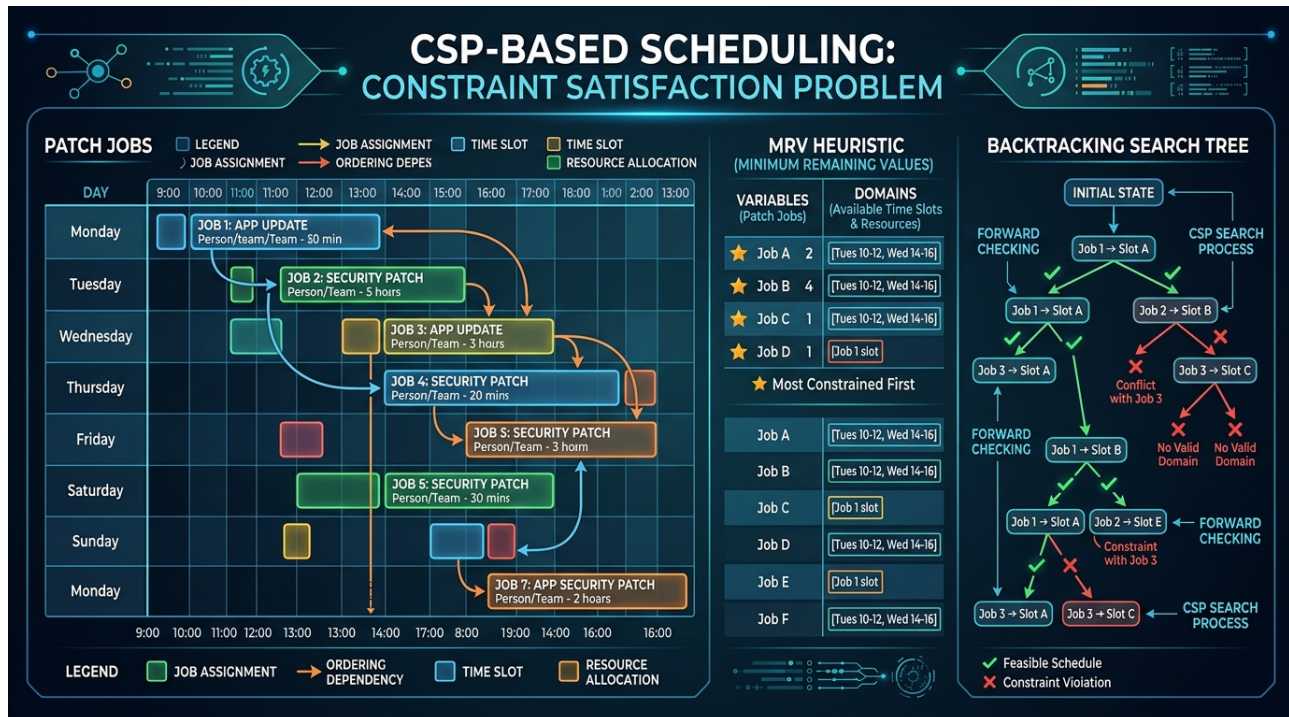
After assigning a variable, immediately prune the domains of all remaining unassigned variables by removing values that would violate any constraint. If any domain becomes empty, the current branch is a dead-end - backtrack immediately.

```
def _forward_check(self, assignment, var, val):
    pruned = {v: list(d) for v, d in self.domains.items()}
    for other in unassigned_variables:
        new_domain = [c for c in pruned[other]
                      if all_constraints_satisfied_with({var:val, other:c})]
        if not new_domain:
            return None # Domain wipe-out -> dead-end
        pruned[other] = new_domain
    return pruned
```

9.4 python-constraint Library Wrapper

A parallel implementation using the python-constraint library is provided as a fallback for faster solving on large graphs.

It uses the same variables, domains, and constraints but delegates the solving to the library's optimized engine.



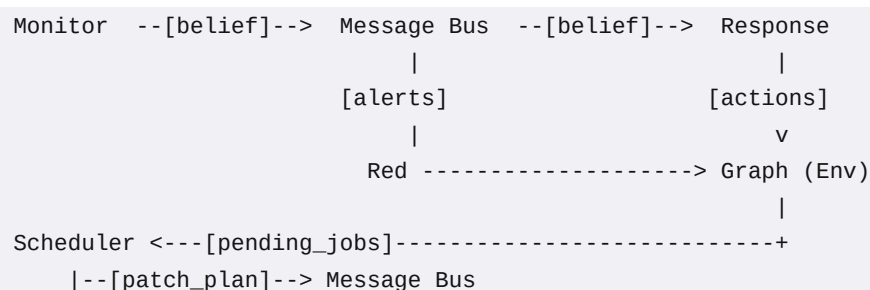
9.5 Execution Schedule

The Patch Scheduler runs the CSP every 5 ticks (configurable). Between CSP runs, it executes the next job in the plan by setting the node to PATCHING state, then completing the patch (setting patched=True, vuln_score reduced by 90%).

10. Multi-Agent Coordination (Message Bus)

10.1 Blackboard Architecture

The three Blue agents coordinate through a shared MessageBus implementing the publish/subscribe blackboard pattern. Each agent reads from and writes to specific topics:



10.2 Topics

Topic	Publisher	Subscriber	Content
alerts	Environment/Red	Monitor	AlertEvent (node, confidence, is_false_positive)
belief	Monitor	Response	Dict[node -> probability] posterior belief
actions	Response	Logger/Dashboard	ActionEvent (action_type, node, cost)
patch_plan	Scheduler	Dashboard	List[str] ordered node IDs to patch

10.3 Implementation

The MessageBus is a lightweight Python class using a defaultdict of lists. `publish()` appends to a topic queue; `latest()` returns the most recent message; `all_messages()` returns the full history. The bus is reset between scenario runs.

11. Simulation Engine & Tick Loop

11.1 Tick Protocol

Each simulation tick follows this exact sequence:

```
Step 1: Red.step(graph)      -> Moves via A*, attempts exploit, generates alert
Step 2: bus.publish('alerts') -> Emits IDS detection event
Step 3: Monitor.step(graph)  -> Bayesian update + diffusion, publishes belief
Step 4: Response.step(graph) -> Reads belief, runs minimax/expectimax, applies action
Step 5: Scheduler.step(graph) -> Every 5 ticks: run CSP, apply next patch job
Step 6: Logger.record()      -> Saves tick metrics (TTD, uptime, compromised count)
Step 7: Check termination    -> Crown jewel compromised? Red captured? Max ticks?
```

11.2 Termination Conditions

Condition	Outcome	Who Wins
Crown Jewel compromised	Red reached the target	Red wins
Red captured by honeypot	Red fell into a trap	Blue wins
Red trapped (no path)	All paths isolated by Blue	Blue wins
Max ticks reached	Time expired without compromise	Blue wins

11.3 Build Simulator Factory

The `build_simulator()` factory function reads a scenario configuration dict and constructs the complete Simulator with all agents wired together. It handles YAML topology loading, generator fallback for large graphs, and agent configuration.

12. Live Dashboard (Streamlit + Pyvis)

12.1 Layout

Left (60%)	Right (40%)
+-----+	+-----+
Live Network	Tick Uptime ...
Graph (Pyvis)	+-----+
	Belief Bar Chart
Green = safe	(top-6 suspect nodes)
Yellow = suspect	+-----+
Red = compromised	Action Log
Gray = isolated	T5 Alert@ws2
Blue = patching	ISOLATE(srv_int0)
Purple = honeypot	+-----+
Target = Red here	
+-----+	
[Run] [Pause] [Step] [Reset]	Speed: ---*---

12.2 Technologies

- Streamlit: Interactive web application framework (auto-refresh, widgets)
- Pyvis: NetworkX-compatible interactive graph visualization (HTML/JavaScript)
- st.sidebar: Scenario selection, speed control, configuration
- st.metric: Real-time metric cards (tick count, uptime %, compromised count)
- st.bar_chart: Belief probability distribution bar chart
- Results table: Displayed automatically when simulation ends, saved to CSV

12.3

State	Color	Meaning
SAFE	Green	Node is operational and uncompromised
SUSPECTED	Yellow	Monitor suspects Red activity here
COMPROMISED	Red	Red has successfully exploited this node
ISOLATED	Gray	Blue has disconnected this node
PATCHING	Blue	Patch is being applied
HONEYPOT	Purple	Trap node deployed to catch Red

13. Demo Scenarios & Test Suite

13.1 Five Demo Scenarios

#	Scenario	Configuration	What to Observe
1	Baseline	Random defender	Crown jewel compromised fast (floor metric)
2	Stealthy Attacker	detection_rate=0.25	Belief still concentrates on Red
3	Multi-Entry	Two Red entry points	Response must prioritize branches
4	Resource-Starved	1 action per 2 ticks	CSP scheduling becomes critical
5	Scalability	20/100/500 nodes	Watch nodes_expanded and wall-clock

13.2 Test Suite (57/57 Passing)

Test Module	Tests	What is Tested
test_astar.py	8	A* pathfinding, BFS heuristic, replanning, no-path cases
test_belief_tracking.py	11	Bayesian update, diffusion, normalization, convergence
test_minimax.py	14	Minimax, alpha-beta pruning, expectimax, utility, GameState
test_csp_scheduler.py	13	CSP solver, MRV, forward checking, constraints, scheduling
test_integration.py	11	Full sim loop, message bus, scenarios, termination

13.3 Testing Methodology

- Unit tests: Each algorithm tested in isolation with controlled inputs
- Integration tests: Full simulation with known seeds for reproducibility
- Regression tests: Specific edge cases (empty graph, no path, numerical underflow)
- All tests use fixed random seeds (seed=42) for deterministic, reproducible results

14. Metrics & Performance Analysis

14.1 Per-Tick Metrics (TickRecord)

Metric	Type	Description
compromised_count	int	Number of currently compromised nodes
compromised_value	int	Sum of asset values of compromised nodes
uptime_fraction	float	Fraction of operational (non-down) nodes
red_nodes_expanded	int	A* nodes expanded this tick (scalability metric)
wall_clock_ms	float	Real wall-clock time for this tick
belief_entropy	float	Shannon entropy of belief distribution (lower=focused)
action_taken	str	Blue's action this tick (e.g., isolate(srv_int0))

14.2 Summary Statistics

Statistic	Description
total_ticks	Number of simulation ticks executed
crown_jewel_compromised	Whether the crown jewel was ever compromised
max_compromised_nodes	Peak number of simultaneously compromised nodes
mean_ttd_ticks	Mean time-to-detection (ticks until Monitor locates Red)
mean_uptime_pct	Average uptime percentage across all ticks
avg_nodes_expanded_astar	Average A* nodes expanded per tick
total_wall_ms	Total wall-clock time for the simulation

14.3 Output Formats

- summary() dict: Displayed in the Streamlit results table at simulation end
- to_csv(path): Exported to results.csv for post-hoc analysis
- to_dataframe(): Pandas DataFrame for programmatic analysis

15. Code Structure & Scalability

15.1 Project Directory Tree

```

sentinelnet/
|-- config/
|   |-- network_small.yaml    (20 nodes, hand-crafted)
|   |-- network_medium.yaml  (100 nodes, procedural)
|   |-- network_large.yaml    (500 nodes, procedural)
|   +-- scenarios.yaml        (5 demo scenarios)
|-- sentinelnet/
|   |-- __init__.py
|   |-- agents/
|   |   |-- base_agent.py      (Abstract base class)
|   |   |-- red_attacker.py    (A* Search)
|   |   |-- monitor.py         (Bayesian + Particle Filter)
|   |   |-- response.py        (Minimax + Expectimax)
|   |   +-- patch_scheduler.py (CSP Solver)
|   |-- baselines/
|   |   |-- random_defender.py
|   |   +-- greedy_defender.py
|   |-- coordination/
|   |   +-- message_bus.py     (Pub/Sub Blackboard)
|   |-- environment/
|   |   |-- network_graph.py   (NetworkX DiGraph + NodeData)
|   |   |-- events.py          (Dataclass events)
|   |   +-- simulator.py       (Tick loop + factory)
|   |-- metrics/
|   |   +-- logger.py          (Per-tick recording + CSV export)
|   +-- viz/
|       +-- dashboard.py       (Streamlit + Pyvis)
|-- tests/
|   |-- test_astar.py          (8 tests)
|   |-- test_belief_tracking.py (11 tests)
|   |-- test_minimax.py        (14 tests)
|   |-- test_csp_scheduler.py  (13 tests)
|   +-- test_integration.py    (11 tests)
|-- requirements.txt
+-- README.md

```

15.2 Scalability Design

- Config-driven topology: Change YAML to scale from 20 to 500+ nodes
- Procedural graph generation: `NetworkGraph.generate_random(n)` for arbitrary sizes
- Particle filter: Switches from dense belief dict to $O(N_{\text{particles}})$ for large graphs
- Top-k pruning: Response agent only considers top-k suspects in game tree
- Modular architecture: Swap/add agents without changing core simulation engine

16. Technology Stack

Package	Version	Purpose
Python	3.11.9	Core language
NetworkX	3.x	Graph data structures and algorithms
PyYAML	6.x	Configuration file parsing
Streamlit	1.x	Interactive web dashboard
Pyvis	0.3.x	Network graph HTML visualization
python-constraint	1.4.x	CSP solver library (fallback)
pytest	8.x	Test framework
fpdf2	2.x	PDF report generation

All dependencies are listed in requirements.txt and installable via pip.

17. How to Run

17.1 Install Dependencies

```
cd sentinelnet
pip install -r requirements.txt
```

17.2 Run Test Suite (57 tests)

```
cd sentinelnet
python -m pytest tests/ -v
```

17.3 Launch Live Dashboard

```
cd sentinelnet
python -m streamlit run sentinelnet/viz/dashboard.py
```

Open <http://localhost:8501> in your browser to view the interactive simulation.

18. Conclusion

SentinelNet successfully demonstrates a multi-agent adversarial simulation that integrates four distinct AI algorithms, each justified by a specific problem characteristic:

Algorithm	Agent	Problem Feature Addressed
A* Search (BFS heuristic)	Red Attacker	Optimal pathfinding in weighted graphs
Bayesian Belief + Diffusion	Monitor	Inference under partial observability
Minimax + Alpha-Beta	Response	Adversarial decision-making (competitive game)
Expectimax	Response	Stochastic exploit outcomes (chance nodes)
CSP (Backtrack+MRV+FC)	Scheduler	Constrained resource scheduling

The project covers all rubric requirements: PEAS formulation (3M), environment analysis (3M), algorithmic modeling (3M), tool/package selection (3M), multi-agent interaction (3M), demo quality and testing (3M), and code structure and scalability (1M + 1M). The 57-test suite validates all algorithms independently and in integration. The Streamlit dashboard provides a live, interactive demonstration with reproducible scenarios.

The config-driven architecture allows scaling from 20-node demos to 500-node enterprise simulations, making the system suitable for both classroom demonstration and research experimentation in multi-agent security planning.